

Advanced Classwork: Decision Arcade

For fast finishers | 100 points + 10 bonus | No new syntax beyond today's lesson

Student name

Date

Part 1 - Silent Trace Challenge

20 points

Do not run the code yet. Trace it by hand. Write the one message that prints for each score.

```
let score: number = 76;

if (score >= 95) {
  console.log("Mythic");
} else if (score >= 85) {
  console.log("Elite");
} else if (score >= 75) {
  console.log("Veteran");
} else if (score >= 60) {
  console.log("Rising");
} else {
  console.log("Training");
}
```

score = 96

score = 95

score = 94

score = 85

score = 84

score = 76

score = 75

score = 74

score = 60

score = 59

For score 76, explain every condition checked until the ladder stops:

Logic Repair Lab

Advanced classwork continued

Part 2 - The Code Runs, But the Game Is Wrong

20 points

A student wanted four robot battery states. The rules are below, but the ladder is in a bad order.

Rules: 90+ Overcharged, 70+ Ready, 35+ Low Power, lower than 35 Shutdown

```
let battery: number = 92;

if (battery >= 35) {
  console.log("Low Power");
} else if (battery >= 70) {
  console.log("Ready");
} else if (battery >= 90) {
  console.log("Overcharged");
} else {
  console.log("Shutdown");
}
```

What does it print for 92?

What should it print for 92?

Which condition catches 92 too early?

Rewrite the ladder in the correct order:

Constraint

Use only the same if / else if / else structure. Do not add loops, functions,

Boundary Test Designer

Advanced classwork continued

Part 3 - Prove the Repair Works

20 points

Choose test values that prove each rule works. Include each exact boundary and the number just below it.

Test purpose	Value	Expected output
Overcharged boundary		
Just below Overcharged		
Ready boundary		
Just below Ready		
Low Power boundary		
Just below Low Power		
One extra high value		
One extra low value		

Which two tests are the most important, and why?

Build a Harder Decision Game

Advanced classwork continued

Part 4 - Design With Overlapping Conditions

25 points

Create a game ladder where lower conditions would accidentally catch higher values if ordered badly.

Theme

Write your rules in English before coding:

Write your TypeScript ladder:

Explain why your order goes from highest to lowest:

Expert Debug Reflection

Advanced classwork conclusion

Part 5 - Debug Without Guessing

15 points

Answer like a careful programmer. The goal is evidence, not speed.

1. A program compiles. What still might be wrong?

2. How can boundary values reveal an incorrect condition?

3. Why should humans test AI-written decision code?

Bonus - Create a Bug on Purpose

+10 points

Make a wrong-order version of your own ladder. Predict the wrong output, then explain the fix.

Submit to Microsoft Teams AND Ishwari Raut ma'am if your teacher asks for this extension.